

10.4 Null Pointers

We said that the value of a pointer variable is a pointer to some other variable. There is one other value a pointer may have: it may be set to a *null pointer*. A *null pointer* is a special pointer value that is known not to point anywhere. What this means is that no other valid pointer, to any other variable or array cell or anything else, will ever compare equal to a null pointer.

The most straightforward way to "get" a null pointer in your program is by using the predefined constant `NULL`, which is defined for you by several standard header files, including `<stdio.h>`, `<stdlib.h>`, and `<string.h>`. To initialize a pointer to a null pointer, you might use code like

```
#include <stdio.h>

int *ip = NULL;
```

and to test it for a null pointer before inspecting the value pointed to you might use code like

```
if(ip != NULL)
    printf("%d\n", *ip);
```

It is also possible to refer to the null pointer by using a constant 0, and you will see some code that sets null pointers by simply doing

```
int *ip = 0;
```

(In fact, `NULL` is a preprocessor macro which typically has the value, or replacement text, 0.)

Furthermore, since the definition of "true" in C is a value that is not equal to 0, you will see code that tests for non-null pointers with abbreviated code like

```
if(ip)
    printf("%d\n", *ip);
```

This has the same meaning as our previous example; `if(ip)` is equivalent to `if(ip != 0)` and to `if(ip != NULL)`.

All of these uses are legal, and although I recommend that you use the constant `NULL` for clarity, you will come across the other forms, so you should be able to recognize them.

You can use a null pointer as a placeholder to remind yourself (or, more importantly, to help your program remember) that a pointer variable does not point anywhere at the moment and that you should not use the "contents of" operator on it (that is, you should not try to inspect what it points to, since it doesn't point to anything). A function that returns pointer values can return a null pointer when it is unable to perform its task. (A null pointer used in this way is analogous to the `EOF` value that functions like `getchar` return.)

As an example, let us write our own version of the standard library function `strstr`, which looks for one string within another, returning a pointer to the string if it can, or a null pointer if it cannot. Here is the function, using the obvious brute-force algorithm: at every character of the input string, the code checks for a match there of the pattern string:

```
#include <stddef.h>

char *mystrstr(char input[], char pat[])
{
```

```

char *start, *p1, *p2;
for(start = &input[0]; *start != '\0'; start++)
{
    /* for each position in input string... */
    p1 = pat;          /* prepare to check for pattern string there */
    p2 = start;
    while(*p1 != '\0')
    {
        if(*p1 != *p2) /* characters differ */
            break;
        p1++;
        p2++;
    }
    if(*p1 == '\0')    /* found match */
        return start;
}

return NULL;
}

```

The `start` pointer steps over each character position in the `input` string. At each character, the inner loop checks for a match there, by using `p1` to step over the pattern string (`pat`), and `p2` to step over the input string (starting at `start`). We compare successive characters until either (a) we reach the end of the pattern string (`*p1 == '\0'`), or (b) we find two characters which differ. When we're done with the inner loop, if we reached the end of the pattern string (`*p1 == '\0'`), it means that all preceding characters matched, and we found a complete match for the pattern starting at `start`, so we return `start`. Otherwise, we go around the outer loop again, to try another starting position. If we run out of those (if `*start == '\0'`), without finding a match, we return a null pointer.

Notice that the function is declared as returning (and does in fact return) a pointer-to-char.

We can use `mystrstr` (or its standard library counterpart `strstr`) to determine whether one string contains another:

```

if(mystrstr("Hello, world!", "lo") == NULL)
    printf("no\n");
else
    printf("yes\n");

```

In general, C does not initialize pointers to null for you, and it never tests pointers to see if they are null before using them. If one of the pointers in your programs points somewhere some of the time but not all of the time, an excellent convention to use is to set it to a null pointer when it doesn't point anywhere valid, and to test to see if it's a null pointer before using it. But you must use explicit code to set it to `NULL`, and to test it against `NULL`. (In other words, just setting an unused pointer variable to `NULL` doesn't guarantee safety; you also have to check for the null value before using the pointer.) On the other hand, if you know that a particular pointer variable is always valid, you don't have to insert a paranoid test against `NULL` before using it.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail feedback](#)